Android Application Security Assessment

Katrina's Mystic Visitors Guide — Salem (LocationMapApp v1.5)

Anti-piracy program OMEN-025, Phase 1 — implementation & threat review Date: 24 May 2026 · **Status:** core implemented & unit-tested; live activation pending three operator credentials **Operating entity:** Destructive AI Gurus, LLC · **Package:** `com.destructiveaigurus.katrinasmysticvisitorsguide` **Prepared for:** operator / counsel review · **Companion reference:** `AndroidSecurity.md` (engineering quick-reference) and `docs/plans/OMEN-025-anti-piracy-phase1.md` (session plan)

Executive summary

Katrina's Mystic Visitors Guide is a **$19.99 flat, fully-offline** paid Android app. Sold this way, with nothing else done, it is trivially copyable: a buyer (or anyone they hand the files to) can extract the installed application package and its content, sideload it onto another phone, and use it forever without paying. The entire value of the product — its curated Salem history, narration, custom maps, and ghost collection — sits in files on the device.

Anti-piracy program **OMEN-025** addresses this without breaking the offline promise. We add a **one-time online "activation handshake"** in front of the otherwise-offline experience. On first launch — and only first launch — the app proves two things to a small server we control: that this is a **genuine Google Play purchase running on a genuine device** (purchase validation), and that the **content shipped with it has not been altered** (content integrity). Once that single check passes, the app writes a tamper-evident, device-locked receipt and runs **offline forever**. Every subsequent launch verifies that local receipt in a few milliseconds with zero network use.

The honest design goal, set by the operator, is to **stop roughly 99% of casual piracy and accept that a skilled reverse-engineer (the remaining ~1%) can eventually defeat any client-side scheme.** This document explains, in plain language, what we have built, the protection it actually provides, the ways it could still be attacked, what we still need to turn it on, the staged test plan as those pieces arrive, and a frank assessment of how well it meets the 99% goal.

Where we stand today: the cryptographic engine, the device-binding, the content-integrity verifier, the server logic, and the decision state-machine are all **written and covered by 71 automated tests** (46 on the Android side, 25 on the server). What remains is **not more**

engineering — it is three credentials only the operator can obtain, and the final wiring + on-device validation that those credentials unlock.

Part 1 — What we have implemented

The protection is built in **two layers** plus the **handshake** that ties them together and the **offline lockdown** they sit inside. Everything below is implemented and unit-tested unless explicitly flagged as pending.

1.1 The offline foundation (already shipping in V1)

Before OMEN-025, the app was already engineered to make **zero network contact** — no analytics, no map-tile fetching, no live data, nothing but the GPS satellite signal. This is enforced four ways at once: a compile-time flag (`V1_OFFLINE_ONLY`), a network interceptor that blocks every outbound request before it reaches the wire, the **removal of the `INTERNET` permission from the release app entirely**, and call-site guards on the dormant features. This is the posture OMEN-025 has to carefully open a single, narrow door in — and then prove the door didn't let anything else through.

1.2 Layer 1 — Purchase validation (is this a real, paid copy on a real phone?)

Google Play Integrity. On first run, the app asks Google's Play Integrity service to vouch for it. Google returns a signed verdict that answers three questions we care about:

- `appLicensingVerdict = LICENSED` — did this Google account actually obtain the app from Google Play? A pirate who copies the files onto a new device has never purchased it, so this comes back unlicensed.
- `appRecognitionVerdict = PLAY_RECOGNIZED` — is this the unmodified app as distributed by Play, or a repackaged/re-signed copy? Tampered or sideloaded copies fail here.
- `deviceIntegrity` **includes** `MEETS_DEVICE_INTEGRITY` — is this a genuine Android device that passes Google's integrity checks (not a bare emulator or a compromised environment)?

Critically, **we do not trust the app's own claim about this verdict.** The app cannot evaluate the verdict itself, because a pirate controls the app. Instead, the encrypted verdict token is sent to **our server**, which asks Google to decrypt and confirm it. The decision is made off-device, where the attacker has no reach.

The hardware-backed device key. When the app first runs, it generates a cryptographic signing key **inside the phone's secure hardware** — the StrongBox secure element where the device has one, otherwise the Trusted Execution Environment (an isolated processor that even a rooted phone cannot easily extract keys from). The key is an EC P-256 key, it is **non-exportable** (the private half physically cannot leave the secure hardware), and it is **device-**

bound but not tied to the user's fingerprint or PIN — so it survives reboots and OS updates but is destroyed by a factory reset, an uninstall, or being moved to a different phone.

The activation receipt. After a passing verdict, the app writes a small "receipt" recording: the device key's fingerprint, a hash of the device's Android ID, which app store installed it, the app's own signing-certificate fingerprint, the content hash, and the verdict. This receipt is stored in encrypted preferences **and** is **signed by the hardware device key**. That signature is the real lock: on every later launch the app reloads the receipt and verifies the signature against the on-device secure element. If the receipt was copied from another phone — or edited in any way — the signature does not verify, and the app refuses to treat the copy as activated.

1.3 Layer 2 — Content integrity (has the shipped content been tampered with?)

At build time, a script (`sign-content-manifest.js`) computes a **SHA-256 hash of each piece of in-scope content** — most importantly `salem_content.db`, the database holding all the paid POIs, narration, and witch-trials material — and assembles them into a **manifest**. That manifest is then **signed with an RSA-2048 private key that lives offline on the operator's machine and is never shipped or committed.**

The app embeds only the matching **public** key. At runtime it verifies the manifest's signature against that public key, then re-computes the hashes of the files actually on disk and compares them to the signed manifest. The security property is subtle but important: a bare checksum stored in a database can simply be patched by anyone who edits the database. A **signed** manifest cannot — forging a new valid signature requires the offline private key, which the attacker does not have. So if someone swaps in a modified content database (say, to inject different POIs or strip a watermark), the signature-anchored check detects it.

Because hashing hundreds of megabytes on a budget phone every launch would be too slow, we use **tiered hashing**: the high-value, small `salem_content.db` (~5 MB) and a couple of small files are fully hashed on **every** launch; the bulky 350 MB map-tile database and image folders are recorded by size/presence and fully checked only at activation time. The map tiles and images are bulk, low-tamper-value assets, so this is a deliberate, documented trade-off (full per-file hashing of those is reserved for Phase 2's encryption).

1.4 The handshake — binding the two layers together

The two layers are stitched together so that a passing result provably concerns **this content on this device at this moment**:

1. The app asks our server for a **nonce** — a short-lived, unguessable, single-use-ish token that proves the request is fresh.

2. The app asks Play Integrity for a verdict token, **cryptographically bound** to a value computed from the nonce and the content hash (`requestHash = SHA-256(nonce + manifestHash)`).
3. The app sends the token, the content hash, and the nonce to our server.
4. Our server re-derives that binding value, asks Google to decrypt the token, and confirms the verdict **and** that the token was bound to our fresh nonce and our expected content. Only then does it return success.

This binding is what stops an attacker from replaying an old genuine token, or pairing a genuine token with tampered content. The exact byte-level recipe for that binding is implemented identically on both sides and **pinned by an automated test** that asserts the Android code produces the same value as the server code (a real cross-checked value, not a guess) — so the two halves can never silently drift apart.

1.5 The server (a stateless Cloudflare Worker)

Our server is a small **Cloudflare Worker** — code that runs on Cloudflare's edge — that we own, in our own repository, deployed to a plain `*.workers.dev` address. By deliberate design it is **stateless and retains nothing**: it has no database, and it does not log the integrity token, the device's IP, or any device identifier. It mints the nonce using a keyed hash (so it can validate the nonce later without storing it), calls Google to decrypt the verdict, checks all the conditions, and returns a yes/no with a coarse reason code. Rate-limiting against abuse is handled by Cloudflare's built-in rules rather than by storing state. This posture matters both for privacy (almost nothing to disclose, nothing to leak) and for the legal/store-listing disclosures.

1.6 The gate and the "no grace window" rule

When finished, the flow inserts a brief **activation screen** between the splash screen and the map. A genuine, already-activated install blows through it in milliseconds (local receipt check, no network). A first-run install with no internet is, by the operator's explicit choice, **hard-blocked** — it shows a "needs internet to activate" screen with a Retry button and does not open the tour. There is no offline grace period that a pirate could exploit by simply staying offline. Debug builds bypass the whole thing (they are sideloaded and have no real purchase verdict), but that bypass is compiled **out** of release builds, which we verify at the binary level before shipping.

Part 2 — What protection this actually gives us

Translating the machinery into plain outcomes:

- **A copied app on a new phone cannot activate.** It was never purchased through that device's Play account (`LICENSED` fails) and/or it is not the recognized Play binary

(`PLAY_RECOGNIZED` fails). With the hard-block rule, an unactivated app is an app that never opens the tour.
- **A copied *receipt* cannot rescue a copied app.** The receipt is signed by secure hardware unique to the original phone. Lift the encrypted preferences to another device and the signature no longer verifies — the copy is treated as never-activated.
- **Tampered content is detected.** Swapping or editing `salem_content.db` (the crown-jewel content) breaks the signed-manifest check on every launch. The attacker cannot forge a new valid signature without the offline private key.
- **The verdict cannot be faked by the app.** Because the verdict is decrypted and judged on our server using Google's key, a pirate who patches the client to "always say licensed" achieves nothing — the client's opinion is never trusted.
- **Replay and content-swap are bound out.** The nonce + content-hash binding means a genuine token captured once cannot be reused later or paired with different content.
- **The offline promise is preserved.** After one activation, the app makes zero network contact forever; the dormant network features stay locked even though the `INTERNET` permission is technically present again.
- **Privacy and disclosure burden stay minimal.** The server retains nothing, so there is almost nothing to disclose, subpoena, or breach.

The cumulative effect: the **easy, zero-skill copy paths — the ones the overwhelming majority of casual pirates would ever attempt — simply do not work.**

Part 3 — Concerns: how this could still be exploited

A security document that only lists strengths is marketing, not engineering. Here are the real limitations, by severity.

3.1 The fundamental ceiling: it is a client-side scheme

Everything that runs on a device the attacker fully controls can, with enough effort, be defeated. A skilled reverse-engineer can disassemble the app, locate the activation gate, and patch it to skip the handshake or to treat itself as always-activated. **This is the explicit, accepted ~1%.** No client-side anti-piracy scheme on any platform is immune to this; Google, Netflix, and every premium app live with the same ceiling. Our job is to make the casual path fail and the skilled path expensive — not to claim impossibility.

What raises the cost for that 1%: the gate is not a single boolean to flip — defeating it means stripping the activation check **and** keeping the content-integrity verifier happy **and** doing so on a build that no longer matches the Play-distributed binary (so it loses `PLAY_RECOGNIZED` and any future server-side checks). Phase 2 (content encryption) raises this dramatically — see Part 6.

3.2 Content is decrypted-at-rest in Phase 1 (the `adb pull` hole)

In Phase 1, the content files (`salem_content.db`, the tile database, images) ship **in the clear** on disk. We detect tampering, but we do not prevent **extraction**. A technically-minded user can pull the content files off the device and read them directly, bypassing the app entirely. The signed manifest stops *modification*, not *copying of the raw assets*. **Closing this is the entire point of Phase 2 (content encryption)**, where the content ships encrypted and the decryption key is only released by the server on a passing verdict, decrypted into app-private storage. Until Phase 2, motivated extraction of the raw data is possible.

3.3 The activation receipt is single-device, but device-cloning is a grey area

The receipt is bound to the secure-hardware key and the Android ID. On a normally-functioning device this is solid. On a **rooted device with advanced cloning tooling**, an attacker might attempt to virtualize or relocate the secure-element-backed key — but StrongBox/TEE keys are specifically designed to resist this, and such a device would typically also fail `MEETS_DEVICE_INTEGRITY`. This is squarely in the skilled-attacker tier.

3.4 Anti-replay is "freshness," not "single-use"

To keep the server stateless and retain-nothing, the nonce is **time-boxed** (valid ~2 minutes) rather than strictly single-use (which would require storing used nonces). In theory, within that short window, a captured genuine token could be replayed. In practice this is low-value: the threat we care about is *copying a paid app to a non-paying device*, which fails the licensing/recognition verdict regardless of nonce reuse, and the receipt that results is still device-key-bound. This is a **recorded, deliberate trade-off** (OMEN note NOTE-L023 #4), not an oversight.

3.5 No certificate pinning (deliberately)

We chose **not** to pin the server's TLS certificate. Pinning would marginally raise the bar against a man-in-the-middle on the activation call, but a stale or rotated pin has a catastrophic failure mode: it would **brick activation for every new buyer** until an app update shipped. For a stateless verdict endpoint protected by standard HTTPS certificate validation, the operator and OMEN judged the bricking risk to outweigh the modest MITM benefit. The defense instead lives in an exact-host / HTTPS-only / no-redirect guard plus a runtime check of the app's own signing certificate.

3.6 Availability dependence on first run only

Activation requires our Cloudflare Worker and Google's Play Integrity service to be reachable **once**, at first run. If either is down at that moment, a genuine new buyer is hard-blocked until they retry with connectivity. This is the cost of the "no grace window" decision. It does **not** affect already-activated users (they are offline-forever). Cloudflare Workers and Play Integrity

are both high-availability services, and the staged-rollout `EXPECTED_MANIFEST_HASHES` mechanism lets us avoid self-inflicted outages during content updates.

3.7 Emulator and debug surface

Debug builds bypass the gate by design. The protection that this bypass is compiled out of release builds rests on the const-folding/R8 behavior — which is why the test plan includes an explicit **binary-level proof** (`apkanalyzer` + a deliberately-tampered release build that must actually lock) rather than an assumption.

Part 4 — What we need next: the tokens

The remaining work is **not engineering** — the code is written and tested. It is **three credentials** that only the operator can obtain, each unlocking a stage of activation. They are independent and can be gathered in any order, but live end-to-end testing needs all three.

4.1 Google Play Console — internal testing track *(the critical-path item)*

Play Integrity verdicts are only meaningful for apps **distributed through Google Play**. We need the app uploaded to a Play Console **internal testing track**, under the **Destructive AI Gurus, LLC** developer account. This is also the longest pole: registering a Play Console developer account triggers a multi-week identity-verification process. **Starting this is the single most time-sensitive operator action.** Until it exists, we cannot obtain a real licensing verdict — only the bypassed debug path can be exercised.

4.2 Google Cloud — Play Integrity API + service account

The app must be linked to a **Google Cloud project** with the **Play Integrity API enabled**, and we need a **service account** with permission to call the verdict-decryption endpoint. That service account's credentials (a JSON file containing a private key) are what our Worker uses to ask Google "is this token genuine, and what does it say?" The JSON is stored **offline** on the operator's machine and loaded into the Worker as secrets — never committed, never shipped. We also need the project's numeric ID wired into the app.

4.3 Cloudflare — a scoped API token

To deploy our Worker, we need a **Cloudflare API token** scoped to deploy Workers, on the operator's Cloudflare account. A plain `*.workers.dev` deployment needs no custom domain or DNS. The token is stored offline and used only at deploy time.

> **One-time keys already in hand:** the offline RSA content-signing key (`~/keys/content-manifest.pem`) and its committed public half already exist — Layer 2 is fully operational today. The three items above are what gate Layer 1's live operation.

Part 5 — The test path as the tokens arrive

The plan is **staged**, so we de-risk continuously rather than in one big-bang at the end. Each stage adds confidence and is independently verifiable.

Stage 0 — Today (no tokens): unit-level confidence ✅ done

- 46 Android unit tests + 25 Worker unit tests, all passing.
- The cross-component cryptographic contracts (content hash, request-hash binding) are pinned to real, mutually-verified values.
- The Worker bundles cleanly and can be exercised locally in a test mode that injects a fake verdict, proving the routing, nonce, binding, and decision logic end-to-end without any Google credentials.

Stage 1 — Cloudflare token arrives: deploy infrastructure

- Configure the Worker's content-hash and package-name variables; set the nonce secret.
- Dry-run and (once Google secrets exist) deploy to `*.workers.dev`; confirm the live nonce endpoint responds.

Stage 2 — Google service account arrives: real verdict decryption

- Load the service-account secrets into the Worker; deploy.
- Confirm the Worker correctly **accepts** a genuine token and **denies** a tampered/sample one, honours nonce expiry, and logs no personal data.
- Wire the real Worker hostname and Google Cloud project number into the app.

Stage 3 — Play Console internal track is live: real licensing + on-device validation

- Build the activation screen and wire it into the launch flow (splash → activation → map), with the defensive redirect that stops anyone from skipping straight to the map.
- Flip the master switch on (in the same change), and validate on real devices (Pixel 8 / Lenovo):
- A genuine internal-track install activates once and then runs offline forever (verified by watching for **zero network sockets** on subsequent cold starts).
- First run in airplane mode is hard-blocked with a working Retry.

- The hardware device-key round-trip works on both StrongBox and TEE devices, and activation completes in well under a second.

Stage 4 — Hardening, binary-proof, and rollout

- Re-add the single `INTERNET` permission to the release app; ship an HTTPS-only network policy.
- **Binary-level proof** with `apkanalyzer`: confirm the permission is present, no cleartext is allowed, and — the important one — that a **deliberately tampered release build actually locks** (proving the debug bypass was truly compiled out, not merely assumed).
- **Regression proof:** confirm the dormant features (weather, transit, aircraft, etc.) stay blocked even though `INTERNET` is now granted.
- Upload to the internal track; run the full end-to-end matrix (hard-block, offline-forever, tamper→lock→recover).

Stage 5 — Disclosures

- Play Data Safety entry (integrity token in transit, **no retention**), a privacy-policy line, store-listing note that a one-time internet connection is required to validate, and registering the new credentials in the OMEN inventory.

Part 6 — The ultimate security of the application when this is complete

When Phase 1 is finished (the work this document describes)

The casual-copy paths are closed: a copied app on a new device cannot activate; a copied receipt cannot rescue it; tampered content is detected; the verdict is judged server-side beyond the attacker's reach; and the app remains fully offline after one validation. This satisfies the operator's stated 99% goal for the launch. The residual exposure is the skilled reverse-engineer (~1%) and the fact that, in Phase 1, the raw content files can still be **extracted** (not modified) by a technical user.

Phase 2 — content encryption (the big upgrade, planned, not yet built)

Phase 2 closes the extraction hole and turns content integrity from *detected* to *enforced*:

- The content ships **encrypted**. The decryption key is released by our server **only** on a passing verdict + matching content hash, over the same activation call.
- Content is decrypted into **app-private storage**, which a normal user cannot reach — closing the `adb pull` hole.

- Tampered or extracted content is no longer merely detectable; it is **cryptographically unusable** — without activation there is no key, and without the key the content is just ciphertext.
- A separate secure-hardware key (distinct from the signing key) handles the encryption, which is why the device-key design already documents that hook.

After Phase 2, defeating the scheme requires not just patching a gate but **obtaining the content-decryption key**, which only ever exists transiently in memory after a genuine, server-blessed activation. This moves the bar from "patch a check" to "extract a key from a live, activated, genuine session" — a materially harder problem.

Phase 3 — residual hardening (optional, the last sliver)

Moving the critical gate into native (NDK) code and adding anti-debugging/anti-instrumentation checks. This chips at the skilled-attacker ~1% but never eliminates it, and is explicitly optional.

Deliberately out of scope, permanently: per-install watermarking/tracing, because it would require retaining identifying data and would break the retain-nothing privacy posture.

Part 7 — Outstanding threats (a frank register)

#	Threat	Phase-1 status	Mitigation / path
1	Casual copy: pull app + sideload to another phone	**Blocked**	Fails `LICENSED` / `PLAY_RECOGNIZED`; hard-block = never opens.
2	Copy app **and** receipt to another phone	**Blocked**	Receipt signed by non-exportable secure-hardware key; signature fails off-device.
3	Edit/swap the content database	**Detected**	Signed manifest; no private key on device to forge a new signature.
4	Patch the client to "always licensed"	**Blocked**	Verdict decrypted/judged on our server, not on-device.
5	Replay a captured genuine token	**Low-value / bound out**	Nonce+content binding; copy still fails licensing; receipt still device-bound. (Freshness, not single-use — accepted residual.)
6	**Extract raw content** (read files, bypass app)	**Open in Phase 1**	Closed by **Phase 2 encryption** (the primary reason Phase 2 exists).
7	Skilled RE: disassemble + patch the gate + satisfy integrity verifier	**Accepted ~1%**	Cost-raised (multi-condition gate, loses `PLAY_RECOGNIZED`); Phase 2/3 raise it further.
8	Rooted-device key cloning	**Skilled-tier**	StrongBox/TEE resistance; such devices typically fail device-integrity.
9	MITM on the activation call	**Standard-HTTPS only**	Host-guard + HTTPS-only + signing-cert self-check; pinning deliberately dropped to avoid bricking buyers.
10	First-run availability (our Worker / Google down)	**By-design dependence**	One-time only; high-availability services; never affects activated users.
11	Emulator / debug bypass leaking to release	**Guarded + proven**	Const-folded out; **binary-level proof** in the test plan, not assumed.

Part 8 — How well does this guard against the 99%?

It helps to think of would-be pirates as tiers, because "99%" is really a statement about **who shows up**, not a single probability.

Tier A — the opportunist (the overwhelming majority). Wants the app without paying; willing to tap "install" on a file someone sent them, or follow a three-step "free APK" blog post. Has **no tooling and no patience.** For this tier, OMEN-025 is **decisive**: the copied app simply will not activate, and with the hard-block it never opens. There is no offline-wait trick, no settings toggle, no obvious file to delete. This tier is **completely stopped** — and this tier is the bulk of real-world casual piracy.

Tier B — the tinkerer. Comfortable with `adb`, rooting, and following intermediate guides, but not writing original exploits. They might successfully **extract the raw content files** in Phase 1 (threat #6) and read the data outside the app — a real but limited loss (they get the data, not a working, updatable app). They will **not** get a functioning activated copy of the application, because that requires defeating the server-side verdict, which their copy-paste tooling cannot do. **Phase 2 closes even the data-extraction path for this tier.** So: partially exposed on raw-data extraction in Phase 1; effectively closed after Phase 2.

Tier C — the skilled reverse-engineer (the ~1%). Can disassemble, patch, and rebuild. Given enough hours, **will** defeat any client-side scheme, ours included. We do not pretend otherwise. What we do is make it **expensive and unrewarding**: they must strip the gate, keep the integrity verifier satisfied, and ship a build that has already lost Play recognition — and after Phase 2, obtain a transient decryption key from a live genuine session. For a $19.99 regional tourism app, that effort-to-reward ratio is poor, which is its own deterrent.

Bottom line. Against the population the operator actually cares about — the casual, mindless copiers who make up the vast majority of piracy attempts — **OMEN-025 Phase 1 is expected to be highly effective, meeting the ~99% goal at launch.** The honest caveats are (a) raw *data* extraction by technically-capable users remains possible until Phase 2, and (b) a determined expert can always break a client-side scheme. Neither undermines the core result: **the simple, no-skill act of copying the app to another phone and using it for free does not work.** Phase 2 then converts "tamper-detected" into "tamper-impossible" and closes the extraction gap, taking the protection from strong-against-casual to strong-against-everyone-but-the-expert.

Part 9 — The complete program plan, session by session

This is the full plan-of-record (mirrors `docs/plans/OMEN-025-anti-piracy-phase1.md`). The program is divided into **Phase 1** (purchase validation + content integrity + handshake + disclosures — the launch-blocking work) and the additive **Phase 2** (content encryption) and **Phase 3** (residual hardening). Status legend: ✅ **done** · □ **to-do** · ⏸ **blocked** on an external credential.

Phase 1 — purchase validation + content integrity

Phase 1 is delivered across eight work sessions. As of 24 May 2026, Sessions 1–5 and the Session-6 *core* are complete and unit-tested; the remaining items are gated on the three operator credentials (Part 4) and the launch-flow wiring those credentials make testable.

Session 1 — Build-time signed content manifest (Layer 2) — ✅ done. Generated the offline RSA-2048 signing keypair; committed only the public half. Wrote the build-time signer that hashes the in-scope content, asserts the database was correctly aligned, signs the manifest, and prints the hash to embed in the server. Extended the asset-verifier to confirm

the signature and hashes. Documented the publish-chain ordering. Tested: signature verifies; a byte-flip fails.

Session 2 — Android foundations: device key + receipt store — ✅ done. Built the hardware-backed device key (StrongBox→TEE fallback, non-exportable, device-bound), the activation-receipt model, and the encrypted + device-key-signed receipt store that rejects copied or edited receipts. Added the necessary build dependencies and code-shrinker keep-rules.

Session 3 — Content-manifest verifier (Android) — ✅ done. The runtime counterpart to the Session-1 signer: loads the embedded public key, verifies the manifest signature over the exact bytes, recomputes the in-scope hashes, and re-derives the canonical hash. Tiered (cheap every-launch vs. full at-activation). Covered by 9 unit tests, including one that pins the Kotlin hash to the real build value.

Session 4 — Activation Worker (`*.workers.dev`) — ✅ steps 1–6 done; deploy ▮. The stateless Cloudflare Worker: a frozen endpoint contract, the HMAC nonce module, the Play Integrity decode + pure verdict-evaluation module, and the router — all retain-nothing, no PII logging, native rate-limiting. Covered by 25 unit tests that run without any Google credentials (a test mode injects a decoded verdict). **Remaining:** the live deploy, which needs the Cloudflare token + Google service account.

Session 5 — Activation network path (Android) — ✅ steps 1–3 done; live test ▮. The fails-closed host guard (exact-host / HTTPS-only / no-redirect, no pinning), the dedicated network client that deliberately omits the offline gate, and the transport API with its full response-mapping matrix. Covered by 15 unit tests. **Remaining:** the fails-closed test against the *deployed* Worker.

Session 6 — Play Integrity + state machine + gate UI — ✅ core done; UI/wiring deferred ▮. The Play Integrity client (with the request-hash binding pinned byte-for-byte to the Worker), the full activation state machine (the pure decision logic for the offline-launch check and the handshake-result mapping, plus the injected orchestration), and the debug-bypass / tripwire-log-only / master-switch flags. Covered by 22 unit tests. **Deferred (intentionally, with Session 7):** the activation screen UI and the splash→activation→map wiring — these change the launch flow and are only meaningfully testable against a deployed Worker on the internal Play track.

Session 7 — Offline re-scope, hardening, rollout — ▮ (needs Play Console). Re-add the single `INTERNET` permission to the release app; ship an HTTPS-only network policy; **prove at the binary level** that the debug bypass is compiled out (a deliberately-tampered release build must actually lock); confirm the dormant network features stay blocked; upload to the internal track; run the full end-to-end matrix.

Session 8 — Disclosures (counsel/operator) — □. Play Data Safety entry (integrity token in transit, no retention), a privacy-policy line, a store-listing note about the one-time

validation, target-audience setting, and registering the new credentials in the OMEN inventory.

Verification matrix (mapped to the session that closes each)

Item	Closes in	How it's proven
Signed manifest catches tampering	S1 ✅	Byte-flip an asset → verifier rejects.
Device key sign→verify; copied receipt rejected	S2/S3	On-device round-trip (StrongBox/TEE); receipt moved to another device fails to load.
Manifest verifier rejects bad/stale/flipped	S3 ✅	Unit tests (sig-invalid / byte-flipped / stale).
Worker denies tampered, accepts genuine; no PII	S4	`wrangler dev` /live with sample + genuine tokens; nonce expiry honored.
Network path fails closed off-host/cleartext	S5	Host-guard test against the deployed Worker.
Hard-block on no-internet first run	S6/S7	Airplane mode → blocked, Retry works on reconnect.
Offline-forever after one activation	S7	Airplane-mode cold starts reach the map with zero sockets (logcat).
Tamper → lock → recover	S7	Re-sign / change device ID / debugger / emulator → locks → re-handshake recovers.
Dormant features stay blocked despite INTERNET	S7	Weather/transit/aircraft remain off in release.
Debug bypass truly stripped from release	S7	`apkanalyzer` + a tampered release build that actually locks.

Phase 2 — content encryption (planned, additive — not in this build)

Builds directly on the Phase-1 handshake; no rewrite required.

1. A **separate** secure-hardware key (encrypt/decrypt, distinct from the signing key — the hook is already documented in the device-key design).
2. **Build-time AES-256 encryption** of the bundled content; the app ships ciphertext.
3. The Worker **releases the content key** over the existing activation call — **only** on a passing verdict + matching content hash.
4. Content is **decrypted into app-private storage**, closing the raw-extraction hole and making tampered content cryptographically unusable (no key → just ciphertext).
5. Migrate the large 350 MB tile database into the encrypted set; runtime then verifies by decrypting, not by per-launch hashing.

Phase 3 — residual hardening (optional, the last sliver)

1. Move the critical gate into **native (NDK) code**.

2. Add **anti-debugging / anti-instrumentation** checks.

Permanently out of scope: per-install **watermarking / tracing** — it would require retaining identifying data and would break the retain-nothing privacy posture.

Prepared 24 May 2026 for operator/counsel review. Engineering quick-reference: `AndroidSecurity.md`*. Authoritative session plan:* `docs/plans/OMEN-025-anti-piracy-phase1.md`*. OMEN-025 threat model:* `~/Development/OMEN/architecture/lma-anti-piracy-assessment.md`*. This assessment describes implemented-and-tested code plus a credential-gated path to live operation; it is not a claim that activation is live as of this date.*